

HTW Dynamic Maze: strategie JEPA, A* distillation et bootstrap WM

HackTheWorld(s) – EB-JEPA

June 20, 2026

1 Objectif

Le pivot *Dynamic Maze* transforme le benchmark maze statique en probleme partiellement observe et stochastique. Le labyrinthe contient des portes qui s’ouvrent et se ferment aleatoirement; l’agent ne voit qu’une fenetre locale (*fog-of-war*). Le but est de montrer qu’un world model (WM) peut exploiter la dynamique apprise du monde, plutot que seulement optimiser une distance brute dans l’espace latent.

Deux familles de methodes sont suivies:

1. une baseline forte mais moins pure: entrainer le JEPA sur des trajectoires A* replannifiees, puis distiller des sous-buts $N = 2$ et $N = 4$;
2. une version plus “world model”: collecter des trajectoires par exploration locale sans A*, puis apprendre une value/reachability head par hindsight TD.

2 Structure du monde

Chaque episode est construit ainsi:

1. generer un labyrinthe DFS 21×21 ;
2. choisir des cellules-murs candidates et les transformer en portes stochastiques;
3. initialiser chaque porte ouverte avec probabilite $p_0 = 0.35$;
4. a chaque action, chaque porte toggle avec probabilite $q = 0.04$;
5. rendre une observation locale sous fog-of-war.

L’observation a quatre canaux:

Canal	Nom	Sens
0	agent	point gaussien localisant l’agent.
1	murs visibles	murs/portes fermees visibles sous fog.
2	inconnu	masque des cellules cachees par le fog-of-war.
3	portes visibles	cellules identifiees comme portes quand elles sont visibles.

La probabilite marginale qu’une porte soit ouverte apres t deplacements est:

$$P_t(\text{open}) = \frac{1}{2} + \left(p_0 - \frac{1}{2}\right)(1 - 2q)^t.$$

Avec $p_0 = 0.35$ et $q = 0.04$, la probabilite remonte vers 0.5: environ 46.4% apres 17 deplacements et 49.9% apres 65 deplacements.

3 Donnees et tenseurs

Le loader renvoie des fenetres de trajectoires:

Tenseur	Sens
$\mathbf{x} \in \mathbb{R}^{B \times 4 \times T \times 63 \times 63}$	sequence d'observations dynamic-maze normalisees.
$\mathbf{a} \in \mathbb{R}^{B \times 2 \times T}$	actions cardinales en coordonnees pixel (dr, dc).
$\mathbf{p} \in \mathbb{R}^{B \times 2 \times T}$	positions normalisees de l'agent, utilisees pour le probe et certains labels.
wall_x, door_y	champs dummy pour garder la signature commune du repo.

4 Pseudo-code: generation A*

La premiere version exploite A* comme professeur de donnees. A* est recalcule a chaque pas sur la grille courante, donc il voit les portes ouvertes/fermees au moment present. Il ne predit pas les toggles futurs.

```
function SAMPLE_ASTAR_TRAJECTORY(config, rng):
    maze = DFS_RANDOM_MAZE(config.maze_height, config.maze_width)
    gates = SAMPLE_GATE_CELLS(maze, config.num_doors)
    gate_state[i] ~ Bernoulli(config.door_initial_open_prob)
    agent = start_cell
    goal = goal_cell

    states, actions, locations = [], [], []

    for t in 0 .. config.n_steps - 1:
        obs_t = RENDER_FOG_OBSERVATION(maze, gates, gate_state, agent, goal)
        grid_t = APPLY_OPEN_GATES(maze, gates, gate_state)

        # A* professor: shortest path on the current full grid.
        path_t = ASTAR(grid_t, agent, goal)
        if path_t exists:
            action_t = FIRST_CARDINAL_STEP(path_t)
        else:
            action_t = ZERO_ACTION

        states.append(obs_t)
        actions.append(action_t)
        locations.append(PIXEL_POSITION(agent))

        agent = STEP_ENV(agent, action_t, grid_t)
        for each gate i:
            with probability q: gate_state[i] = not gate_state[i]

    start = RANDOM_SLICE_START(n_steps, sample_length)
    return WINDOW(states, actions, locations, start, sample_length)
```

Dans le code, cette variante correspond a:

Config	Valeur
teacher_policy	oracle_replan
layout_use_astar_filter	true
sample_length	17
n_steps	65 ou 129 selon le run

5 Pseudo-code: entraînement JEPA A*

Le JEPA apprend la dynamique action-conditionnée dans l'espace latent. L'encodeur produit $z_t = f_\theta(x_t)$, le predictif recurrent produit $\hat{z}_{t+k} = g_\phi(z_t, a_{t:t+k-1})$, puis on compare aux latents réels.

```
function TRAIN_JEPA_ASTAR(config):
    loader = STREAM_DATASET(policy = oracle_replan)
    encoder = ImpalaEncoder(input_channels = 4)
    predictor = RNNPredictor(action_dim = 2)
    regularizer = VICReg + temporal similarity + inverse dynamics
    probe_xy = MLP(z -> xy)

    for epoch in 1 .. E:
        for batch (x, a, loc) in loader:
            z_real = encoder(x)
            z_pred = predictor.rollout(z_real[:, t0], actions = a)

            L_pred = MSE(z_pred, stopgrad(z_real_future))
            L_reg = cov/std/sim/idm regularization
            L_probe = MSE(probe_xy(stopgrad(z_real_t0)), loc_t0)

            L_total = L_pred + L_reg
            update(encoder, predictor, regularizer) on L_total
            update(probe_xy) on L_probe

    save encoder, predictor, probe_xy
```

La loss principale est:

$$\mathcal{L}_{pred} = \frac{1}{K} \sum_{k=1}^K \|\hat{z}_{t+k} - \text{sg}(z_{t+k})\|_2^2.$$

Avec les régularisations:

$$\mathcal{L}_{WM} = \mathcal{L}_{pred} + \lambda_{std} \mathcal{L}_{std} + \lambda_{cov} \mathcal{L}_{cov} + \lambda_{sim} \mathcal{L}_{sim} + \lambda_{idm} \mathcal{L}_{idm}.$$

6 Sous-buts A* distilles

Après entraînement du WM, on gèle le JEPA et on entraîne un MLP de sous-but. Pour un instant t , le label est la position N pas plus loin dans la trajectoire observée:

$$y_t = p_{\min(t+N, T)}, \quad \mathcal{L}_{sg} = \|h_\psi(z_t, p_T) - y_t\|_2^2.$$

Pseudo-code:

```
function TRAIN_SUBGOAL_HEAD(frozen_jepa, N):
    for batch (x, a, loc) in A*_trajectory_loader:
        z = frozen_jepa.encode(x)
        goal_xy = loc[:, :, -1]
        label_t = loc[:, :, min(t+N, T-1)]
        pred_t = SubgoalPredictor(z_t, goal_xy)
        update head on MSE(pred_t, label_t)
```

A l'évaluation, A* n'est plus appelé. Le head prédit un waypoint, puis le WM teste les actions cardinales par rollout latent et choisit celle qui se rapproche le mieux du waypoint.

7 Resultats actuels

Run starter:

```
/lustre/work/vivatech-goma/gmeynard/runs/dynamic_maze_jepa_subgoal_starter_20260620_034609
```

Methode	Success	Efficiency	Steps moyens	Blocked moves
fog_optimistic	14.1%	0.134	437.7	0.0
memory_optimistic	100.0%	0.909	79.7	0.0
oracle_replan	100.0%	0.966	70.7	0.0
JEPA subgoal $N = 2$	57.8%	0.403	280.8	103.3
JEPA subgoal $N = 4$	73.4%	0.494	230.7	93.0

Lecture: le WM distille A* bat largement les baselines fog-only, mais reste loin des solveurs symboliques optimistes. Le principal défaut est l'efficacite: il atteint souvent le but, mais tente encore trop de mouvements bloques.

8 Version pure WM: bootstrap sans A*

Pour mieux coller au theme *world models*, la prochaine variante supprime A* de la collecte de donnees et du budget d'évaluation.

Collecte. On utilise `local_goal_frontier`: une politique myope qui ne regarde que la carte visible, les cellules deja visitees et une faible heuristique Manhattan vers le goal. Elle ne lance jamais de graph search.

```
function LOCAL_GOAL_FRONTIER_POLICY(obs, memory):
  for each cardinal action a:
    if local next cell is blocked:
      score[a] = -infinity
    else:
      new_visible = fog_window(next_cell) - known_cells
      revisit = visit_count[next_cell]
      goal_bias = manhattan(next_cell, goal_cell)
      score[a] = novelty_bonus * |new_visible|
                - revisit_penalty * revisit
                - goal_bias_weight * goal_bias
                + small_noise
  with probability epsilon:
    return random_cardinal_action()
  return argmax_a score[a]
```

Value hindsight. Dans une fenetre de trajectoire exploree, tout etat futur devient un pseudo-but. On entraine une value head $V_\eta(z_t, z_g)$ a predire le retour discount vers ce pseudo-but, et on la calibre aussi sur les rollouts imagines du WM.

```
function TRAIN_BOOTSTRAP_WM_VALUE(config):
  loader = STREAM_DATASET(policy = local_goal_frontier,
                           layout_use_astar_filter = false)
  train JEPA dynamics as before

  for batch (x, a, loc) in loader:
    z_real = jepa.encode(x)
    z_roll = jepa.rollout(x_0, a)
```

```

for horizon h in {1,2,4,8,16,32}:
    goal = z_real[:, t+h]
    done = 1 if h == 1 else 0
    target = done + gamma * (1-done) * V_target(z_next, goal)

    L_value += MSE(V(z_real_t, goal), target)
    L_value += MSE(V(z_roll_t, goal), target)

update value head
EMA-update target value head

```

Evaluation. Le controle est A*-free:

```

function EVAL_BOOTSTRAP_VALUE(ckpt):
    for episode in test_episodes:
        obs = env.reset()
        goal_enc = jepa.encode(goal_observation)

        for step in 1 .. fixed_budget:
            for action in cardinal_actions:
                z_future[action] = jepa.rollout(obs, repeat(action, K))
                score[action] = V(z_future[action], goal_enc)
            action = argmax(score - revisit_penalty)
            obs = env.step(action)
            if reached_goal: success

```

9 Pourquoi ce bootstrap est plus defendable

Question	A* distille	Bootstrap WM
Collecte de donnees	professeur A* optimal my-opique	exploration locale sans solveur
Signal de cout	position A* N pas plus loin	reachability/value hindsight
Planification eval	A*-free	A*-free
Budget eval	proportionnel a A* initial	fixe, sans A*
Narration jury	bon controle appris	generalisation par modele du monde

10 Risques

- L'exploration peut ne pas atteindre assez souvent des etats proches du goal.
- La valeur hindsight apprend surtout la connectivite locale si les trajectoires ne couvrent pas de longs chemins.
- Le WM peut apprendre une dynamique locale correcte sans apprendre une strategie globale robuste.
- Le resultat doit donc etre compare a `probe_pos` et `repr_dist` sous le meme budget, pas seulement aux baselines A*.

11 Etat de la run bootstrap

Run bootstrap lancee sur Dalia:

Champ	Valeur
Job Slurm	75812
Noeud	dalianv101
Run dir	/lustre/work/vivatech-goma/gmeynard/runs/dynamic_maze_bootstrap_20260620_0510
Config	train_dynamic_maze_bootstrap.yaml
Collecte	local_goal_frontier, layout_use_astar_filter=false
Budget train	$5 \times 16384 = 81920$ trajectoires nominales
Eval	learned_value, probe_pos, repr_dist, 64 episodes chacun

Etat final observe le 20 juin 2026:

- [x] le job Slurm 75812 est COMPLETED;
- [x] duree totale: 01:32:25 sur un GPU B200;
- [x] le warm-up du pipeline stream est termine;
- [x] la config est chargee et confirmee: `teacher_policy=local_goal_frontier, layout_use_astar_filter=false, value_mode=multi_horizon_td`;
- [x] checkpoints produits jusqu'a `e-4.pth.tar` et `latest.pth.tar`;
- [x] evaluations `learned_value/probe_pos/repr_dist` produites;
- [x] GIFs des premiers episodes copies dans `outputs/dynamic_maze_bootstrap_20260620_051011`.

Objectif	Success	Steps moyens	Blocked moves	Manhattan final
<code>learned_value</code>	70.3%	289.5	88.6	7.0
<code>probe_pos</code>	65.6%	323.2	84.7	6.8
<code>repr_dist</code>	75.0%	244.9	85.1	4.1

Reference A* sequentielle sur la meme distribution non filteree:

Politique	Success	Efficiency	Steps moyens	A* initial	Blocked
<code>oracle_replan_seq_rng</code>	100.0%	0.963	67.5	73.1	0.0

Lecture en pourcentage du cas ideal A*: le `learned_value` atteint 70.3% du taux de succes ideal, mais seulement environ 23.3% de l'efficacite en nombre de deplacements. Le meilleur objectif actuel reste `repr_dist` avec 75.0% de succes et environ 27.6% d'efficacite en deplacements. La value head apprend donc un signal utile, mais ne soutient pas encore completement la claim Track 7 "learned cost > latent distance".

12 Checklist

- [x] Dynamic maze avec portes stochastiques et fog-of-war.
- [x] Baselines A* strictes et GIFs.
- [x] JEPA A* + sous-buts $N = 2, N = 4$.
- [x] Visualisations sans fog des portes et probabilites.
- [x] Politique de collecte `local_goal_frontier` sans A*.
- [x] Config bootstrap `train_dynamic_maze_bootstrap.yaml`.

- [x]Evalueur bootstrap `eval_bootstrap_value.py`.
- [x]Lancer le batch bootstrap sur Dalia.
- [x]Suivre le job 75812 jusqu'au checkpoint WM/value.
- [x]Comparer `learned_value`, `probe_pos`, `repr_dist`.
- [x]Ajouter GIFs et resultats bootstrap dans le depot.
- []Ameliorer `learned_value` pour battre `repr_dist`.